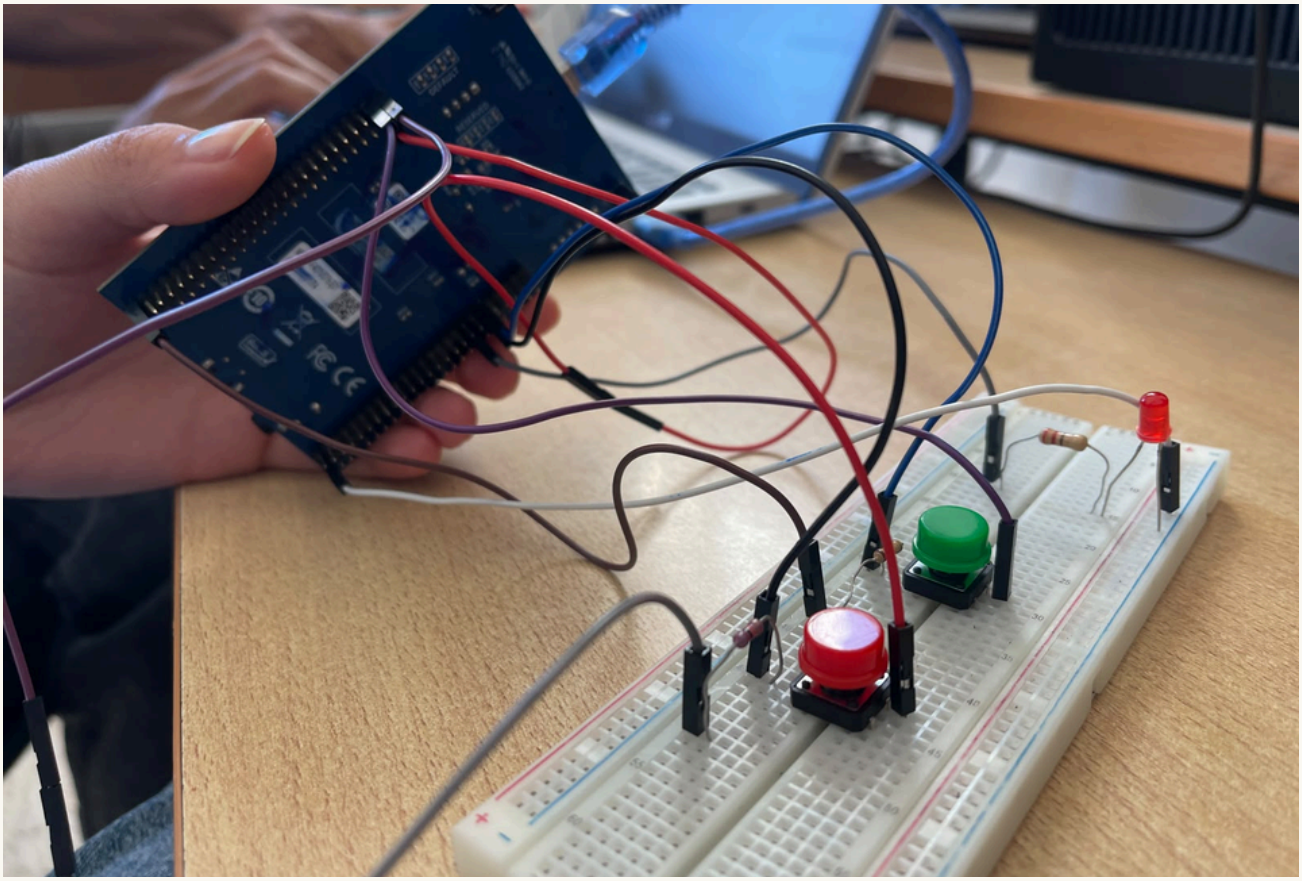
Compterendu 4

groupe 4

PROGRAMMATION CIBLE EMBARQUE

prof:
Ftirich Atef

étudiants:
Fadwa Bougoffa
Islem Guesmi
Mohamed wahib Elfahem



Ce TP a pour objectif d'approfondir la maîtrise de la carte STM32F407VGTx et de la programmation des entrées-sorties numériques (GPIO). Le travail consiste à commander une LED à l'aide de deux boutons poussoirs distincts : le premier bouton permet d'allumer la LED, tandis que le deuxième permet de l'éteindre. Le fonctionnement reste identique même lorsque les boutons sont actionnés plusieurs fois successivement. À travers cette manipulation, nous cherchons à comprendre la lecture de plusieurs entrées numériques, la commande d'une sortie selon les actions de l'utilisateur et la gestion des états d'un système embarqué. Ce TP permet également de renforcer nos compétences en câblage électronique, en configuration du microcontrôleur et en programmation en langage C.

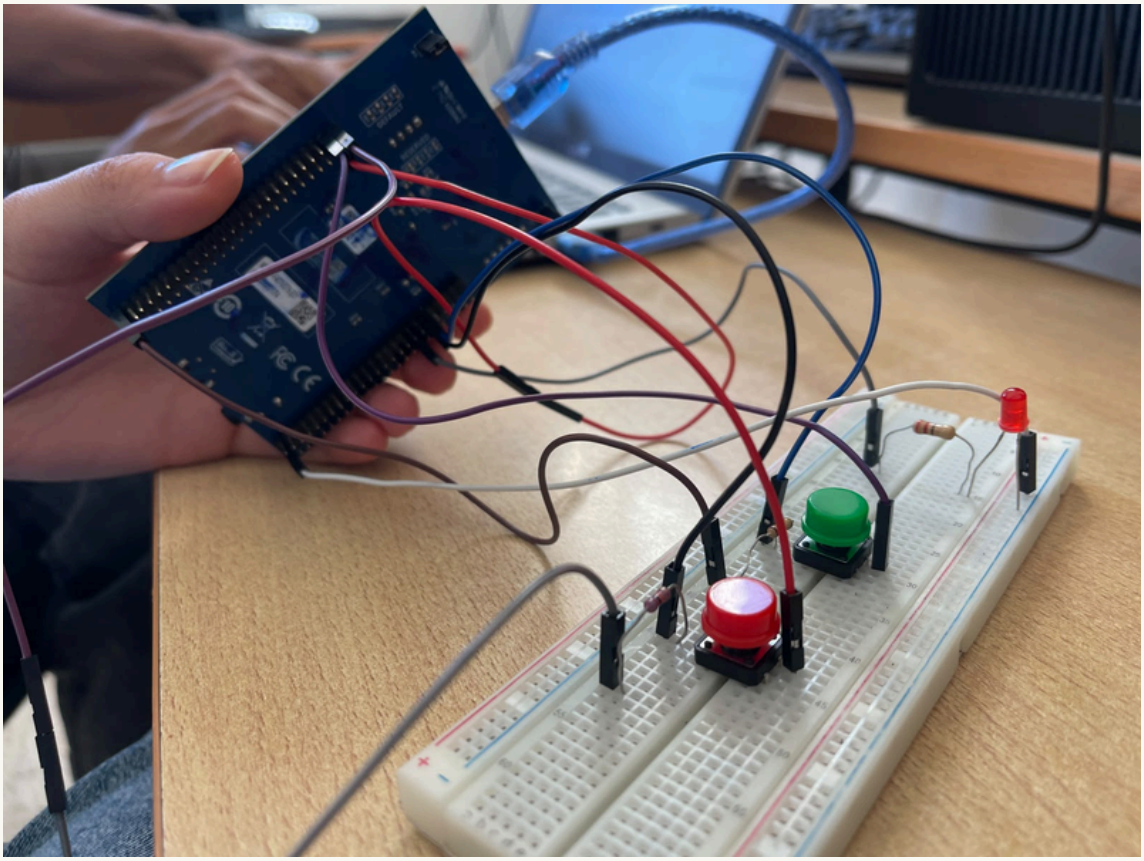
1.1 etape n°1 : Cablage

Dans cette première étape, nous avons réalisé le câblage du circuit électronique en utilisant

- une carte de développement STM32
- une plaque d'essai (breadboard)
- 2 boutons poussoir
- un LED
- des résistances
- des fils de connexion (Dupont).

ce montage comporte deux boutons distincts. Le premier bouton est utilisé pour allumer la LED, tandis que le deuxième permet de l'éteindre. Chaque bouton est relié à une broche d'entrée numérique différente du microcontrôleur STM32, ce qui permet de détecter indépendamment les actions de l'utilisateur.
La LED rouge est reliée à une broche de sortie numérique à travers une résistance de limitation de courant afin de la protéger. Les connexions sont réalisées sur la breadboard à l'aide des fils Dupont, en respectant les broches choisies pour les entrées, la sortie et la masse commune.
Une fois le montage terminé, nous avons vérifié les connexions afin de préparer la configuration des GPIO et la programmation du microcontrôleur.

1.2 figures de cablage



Câblage de deux boutons poussoirs pour commander l'allumage et l'extinction d'une LED à l'aide de la carte STM32.

2.1 Deuxième étape : Parametrage de STM32

Dans STM32CubeMX, nous avons configuré les broches suivantes :

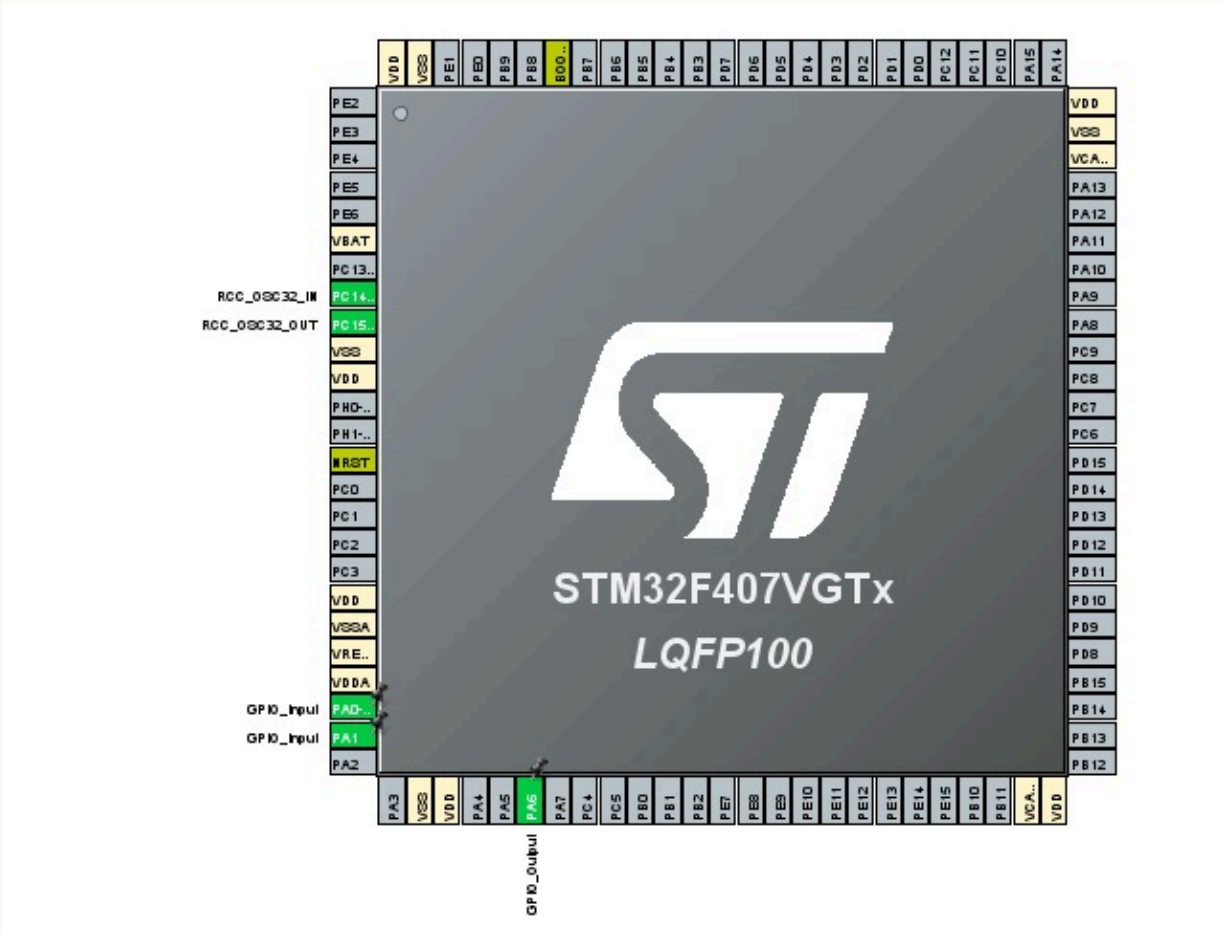
- PA0 : configurée en entrée numérique (GPIO_Input) pour détecter l'appui sur le premier bouton poussoir, qui permet d'allumer la LED.
- PA1 : configurée en entrée numérique (GPIO_Input) pour détecter l'appui sur le deuxième bouton poussoir, qui permet d'éteindre la LED.
- PA5 : configurée en sortie numérique (GPIO_Output) pour commander la LED rouge.

2. Configuration des paramètres GPIO

Dans l'onglet System Core → GPIO, nous avons défini les paramètres de chaque broche selon sa fonction :

Broche	PA0/PA1	PA5
Fonction	1 et 2eme bouton	LED rouge
Mode de configuration	GPIO Input	GPIO Output Push-Pull

Les broches PA0 et PA1 permettent au microcontrôleur de détecter séparément les actions des deux boutons. La broche PA5 commande l'allumage et l'extinction de la LED. La configuration de chaque entrée doit correspondre au câblage réel du bouton et à la présence éventuelle d'une résistance externe de pull-up ou de pull-down.



Parametrage ou configuration de STM32

3.1 Troisieme étape : codage de carte STM32

main.c

```c
#include "stm32f4xx.h"

static void delay_ms(uint32_t ms)
{
    while (ms--)
        for (volatile uint32_t i = 0; i < 4000; i++) __NOP();
}

int main(void)
{
    /* Activer l'horloge GPIOA */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    (void)RCC->AHB1ENR;

    /* PA5 en sortie pour la LED */
    GPIOA->MODER &= ~(3U << (5 * 2));
    GPIOA->MODER |=  (1U << (5 * 2));

    /* PA0 et PA1 en entrées */
    GPIOA->MODER &= ~((3U << (0 * 2)) | (3U << (1 * 2)));

    /* Pas de pull-up/pull-down interne */
    GPIOA->PUPDR &= ~((3U << (0 * 2)) | (3U << (1 * 2)));

    while (1)
    {
        /* Bouton 1 sur PA0 : allumer la LED */
        if (GPIOA->IDR & GPIO_IDR_ID0)
        {
            GPIOA->BSRR = (1U << 5);
```

main.c

```c
    (void)RCC->AHB1ENR;

    /* PA5 en sortie pour la LED */
    GPIOA->MODER &= ~(3U << (5 * 2));
    GPIOA->MODER |=  (1U << (5 * 2));

    /* PA0 et PA1 en entrées */
    GPIOA->MODER &= ~((3U << (0 * 2)) | (3U << (1 * 2)));

    /* Pas de pull-up/pull-down interne */
    GPIOA->PUPDR &= ~((3U << (0 * 2)) | (3U << (1 * 2)));

    while (1)
    {
        /* Bouton 1 sur PA0 : allumer la LED */
        if (GPIOA->IDR & GPIO_IDR_ID0)
        {
            GPIOA->BSRR = (1U << 5);
            delay_ms(20);
        }

        /* Bouton 2 sur PA1 : éteindre la LED */
        if (GPIOA->IDR & GPIO_IDR_ID1)
        {
            GPIOA->BSRR = (1U << (5 + 16));
            delay_ms(20);
        }
    }
}
```

3.2 explication du code

1. Inclusion de la bibliothèque

```
#include "stm32f4xx.h"
```

Cette instruction inclut le fichier d'en-tête de la famille STM32F4. Il permet d'accéder aux registres matériels du microcontrôleur, notamment ceux du contrôleur d'horloge et des ports GPIO.

2. Fonction de temporisation

```
static void delay_ms(uint32_t ms)
{
   while (ms--)
   {
      for (volatile uint32_t i = 0; i < 4000; i++)
         __NOP();
   }
}
```

- **static limite l'utilisation de la fonction au fichier courant.**
- **void indique que la fonction ne renvoie aucune valeur.**
- **uint32_t ms représente le nombre d'itérations de temporisation souhaitées.**
- **while (ms--) répète la boucle tant que la valeur de ms est différente de zéro.**
- **for (...) exécute une boucle interne de 4000 itérations.**
- **volatile empêche certaines optimisations du compilateur sur la variable i.**
- **__NOP() exécute une instruction qui ne réalise aucune opération particulière.**

Cette fonction crée une attente approximative qui permet de ralentir l'exécution du programme. Elle ne garantit pas une durée exacte en millisecondes.

3. Fonction principale

```
int main(void)
{
```

La fonction main() constitue le point de départ du programme. Elle contient les instructions exécutées par le microcontrôleur après son démarrage.

4. Activation de l'horloge GPIOA

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
(void)RCC->AHB1ENR;
```

- **RCC désigne le contrôleur des horloges du microcontrôleur.**
- **AHB1ENR est le registre permettant d'activer les horloges des périphériques concernés.**
- **RCC_AHB1ENR_GPIOAEN correspond au bit d'activation de l'horloge du port GPIOA.**
- **L'opérateur |= permet d'activer ce bit sans modifier les autres bits du registre.**
- **La lecture suivante du registre permet de s'assurer que l'écriture a été effectuée avant de poursuivre.**

Cette étape est nécessaire pour pouvoir utiliser les broches PA0, PA1 et PA5.

5. Configuration de PA5 comme sortie

```
GPIOA->MODER &= ~(3U << (5 * 2));
GPIOA->MODER |= (1U << (5 * 2));
```

Le registre MODER définit le mode de fonctionnement des broches GPIO. Chaque broche utilise deux bits pour déterminer son mode.

- **La première instruction efface les deux bits associés à PA5.**
- **La deuxième positionne ces bits sur 01, ce qui configure PA5 en sortie numérique (GPIO_Output).**

La broche PA5 peut ainsi commander la LED rouge.

6. Configuration de PA0 et PA1 comme entrées

```
GPIOA->MODER &=
 ~((3U << (0 * 2)) | (3U << (1 * 2)));
```

Cette instruction configure les broches PA0 et PA1 en entrées numériques.

- **PA0 reçoit le signal du premier bouton, utilisé pour allumer la LED.**
- **PA1 reçoit le signal du deuxième bouton, utilisé pour éteindre la LED.**
- **L'opérateur | combine les masques des deux broches.**
- **L'opérateur ~ inverse le masque afin d'effacer les bits correspondants.**
- **L'opérateur &= applique le masque au registre sans modifier la configuration des autres broches.**

7. Désactivation des résistances internes

```
GPIOA->PUPDR &=
 ~((3U << (0 * 2)) | (3U << (1 * 2)));
```

Le registre PUPDR permet de configurer les résistances internes de pull-up et de pull-down.

Cette instruction met à zéro les bits de configuration de PA0 et PA1, ce qui désactive les résistances internes. Le montage doit donc utiliser des résistances externes adaptées pour maintenir les entrées dans un état logique défini lorsque les boutons ne sont pas appuyés.

8. Boucle infinie

```
while (1)
{
```

Cette boucle permet au microcontrôleur de surveiller continuellement les deux boutons. Le programme reste actif et peut réagir aux actions de l'utilisateur sans s'arrêter.

9. Détection du premier bouton

```
if (GPIOA->IDR & GPIO_IDR_ID0)
```

- **IDR est le registre permettant de lire l'état logique des broches.**
- **GPIO_IDR_ID0 représente le masque correspondant à PA0.**
- **L'opérateur & vérifie si le bit de PA0 est à 1.**

Lorsque PA0 est à l'état logique haut, la condition est vraie et le programme exécute les instructions suivantes.

10. Allumage de la LED

```
GPIOA->BSRR = (1U << 5);
```

Le registre BSRR permet de modifier l'état des sorties GPIO.

L'écriture du bit 5 dans la partie basse du registre met PA5 à l'état haut (1). La LED s'allume si elle est câblée pour être active à l'état haut.

11. Temporisation après l'appui du premier bouton

delay_ms(20);

Cette instruction introduit une courte attente après la détection du premier bouton. Elle ralentit les répétitions de lecture et peut contribuer à limiter les effets des rebonds mécaniques du bouton. Cependant, le programme ne vérifie pas le relâchement du bouton : un appui maintenu peut donc entraîner plusieurs exécutions de cette instruction.

12. Détection du deuxième bouton

if (GPIOA->IDR & GPIO_IDR_ID1)

Cette condition vérifie si PA1 est à l'état logique haut. Lorsque le deuxième bouton est appuyé, le programme exécute la commande d'extinction de la LED.

13. Extinction de la LED

GPIOA->BSRR = (1U << (5 + 16));

Le registre BSRR possède une partie basse permettant de mettre une sortie à 1 et une partie haute permettant de la remettre à 0.

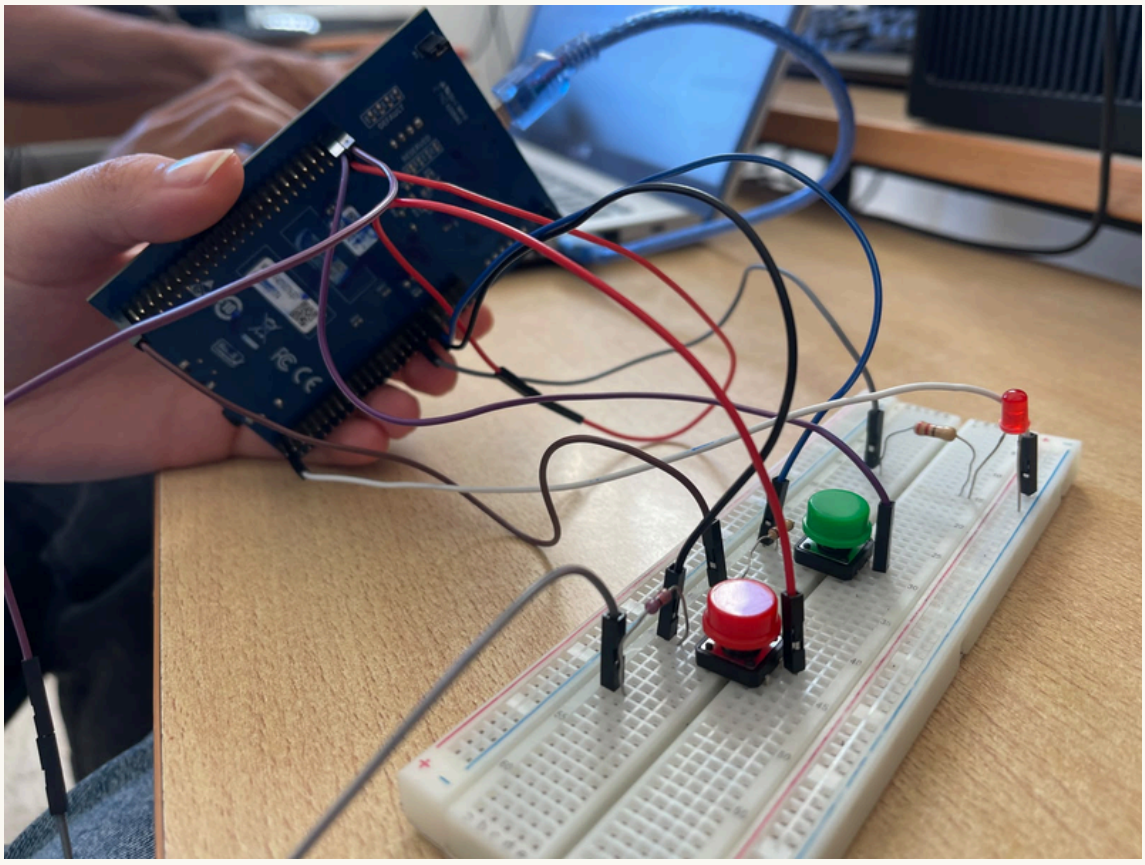
Le décalage de 16 bits positionne le bit correspondant à la remise à zéro de PA5. La LED s'éteint donc lorsque le deuxième bouton est détecté.

14. Temporisation après l'appui du deuxième bouton

delay_ms(20);

Cette instruction ajoute une courte attente après la commande d'extinction. Comme pour le premier bouton, elle ne remplace pas une véritable détection du relâchement.

4 resultat final



conclusion

Au cours de ce TP, nous avons appris à commander une LED à l'aide de deux boutons poussoirs connectés à la carte STM32F407VGTx. Le premier bouton permet d'allumer la LED, tandis que le second permet de l'éteindre. Cette manipulation nous a permis de mettre en pratique la configuration des broches GPIO en entrée et en sortie, ainsi que la lecture de l'état des boutons et la commande de la LED par programmation en langage C. Ce TP nous a ainsi aidés à mieux comprendre l'interaction entre les composants électroniques et le microcontrôleur STM32.